UNIVERSITÉ DE ROUEN NORMANDIE
FACULTÉ DES SCIENCES ET TECHNIQUES

ANNÉE UNIVERSITAIRE 2025–2026

Master 1 – Génie Informatique et Logiciel

Architecture Decision Records

Système de Gestion de Flotte

Contexte applicatif : DHL Express

Semaine 1

14 mars 2026 | Encadrants : Lydia & Luc

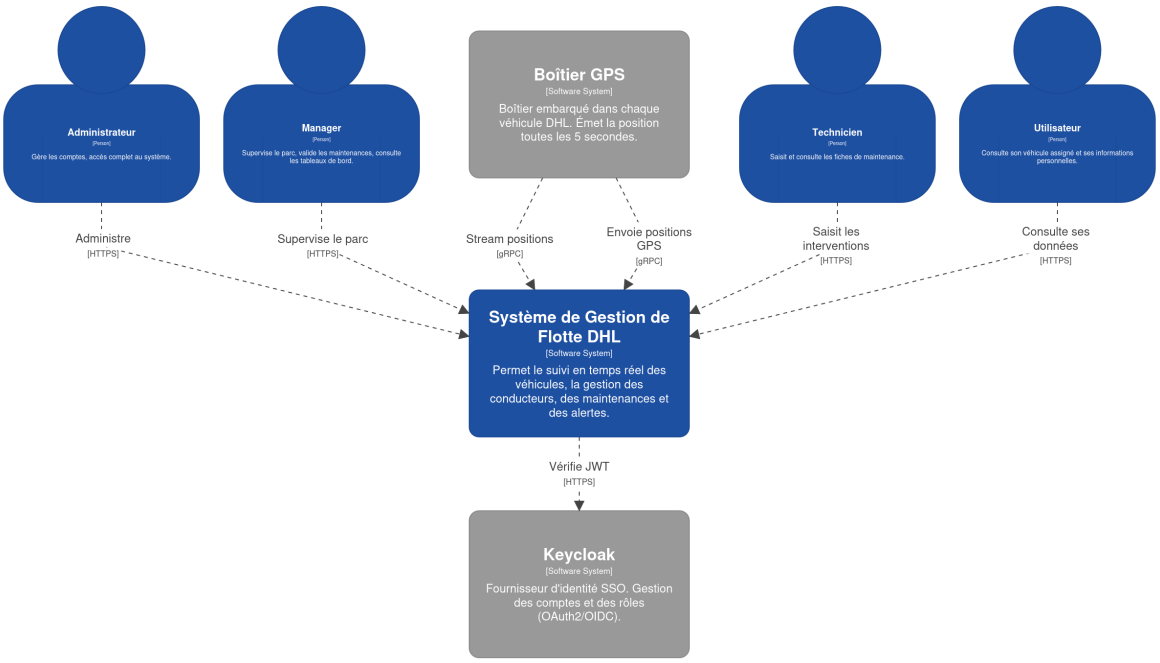
Contents

Architecture Decision Records

Système de Gestion de Flotte — DHL Express

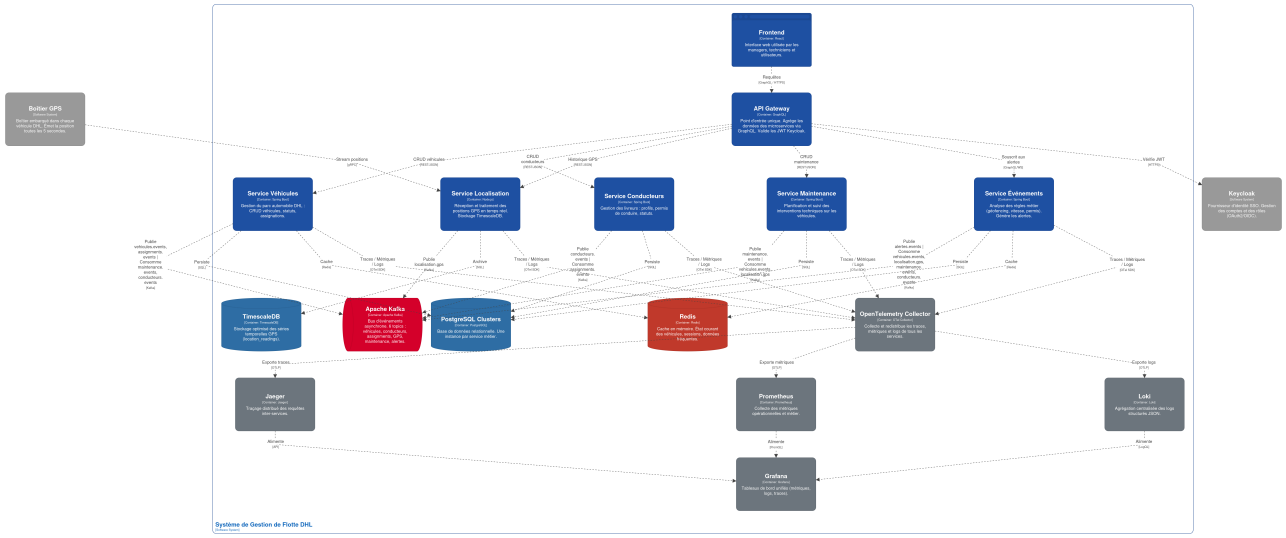
Ce document recense les décisions d'architecture prises en semaine 1 pour le système de **suivi en temps réel de la flotte de véhicules DHL Express**. Le système couvre la gestion des livreurs, l'affectation des véhicules aux tournées, la maintenance du parc et les alertes automatiques. Les utilisateurs sont les **administrateurs**, **managers**, **techniciens** et **utilisateurs**.

C1 — Vue système



C1 — Contexte système : Gestion de Flotte DHL
Vue d'ensemble des acteurs et systèmes externes interagissant avec le système.
samedi 14 mars 2026 à 10:16 heure normale d'Europe centrale

C2 — Architecture des conteneurs



C2 — Conteneurs : Gestion de Flotte DHL
Décomposition interne du système en microservices, bases de données et infrastructure.
samedi 14 mars 2026 à 10:16 heure normale d'Europe centrale

ADR-001 — Architecture microservices

Statut : **Accepté**

Contexte

Le système DHL doit gérer simultanément des responsabilités très différentes : le suivi GPS en temps réel des camionnettes de livraison, la gestion administrative des véhicules et des livreurs, la planification des maintenances, et l'envoi d'alertes aux managers. Ces besoins évoluent à des rythmes différents et ont des contraintes de performance distinctes. Une architecture monolithique rendrait difficile la scalabilité ciblée (par exemple, augmenter uniquement la capacité GPS en période de fêtes) et le déploiement indépendant de chaque partie.

Décision

Le système est découpé en **5 microservices indépendants**, chacun propriétaire de ses propres données et déployable séparément via Kubernetes :

Service	Technologie	Responsabilité
Véhicules	Spring Boot	CRUD du parc, gestion des statuts et de l'historique
Conducteurs	Spring Boot	Profils livreurs, permis de conduire, assignations
Localisation	Node.js	Streaming GPS temps réel, stockage TimescaleDB
Maintenance	Spring Boot	Planification et suivi des interventions techniques
Événements	Spring Boot	Analyse des règles métier, génération des alertes

Conséquences

Avantages : isolation des pannes (une panne du service GPS n'impacte pas la gestion des conducteurs), scalabilité ciblée, déploiements indépendants par service, technologie adaptée à chaque besoin.

Inconvénients : complexité opérationnelle accrue au démarrage, nécessité d'une API Gateway et d'un bus d'événements pour la communication inter-services.

ADR-002 — gRPC pour le streaming GPS

Statut : **Accepté**

Contexte

Les boîtiers GPS embarqués dans les camionnettes DHL émettent leur position toutes les 5 secondes. Pour une flotte de 200 véhicules, cela représente environ 40 messages par seconde en continu. Le protocole REST/HTTP classique, basé sur un modèle requête-réponse, n'est pas adapté à ce flux unidirectionnel permanent et génèrerait une surcharge réseau importante.

Décision

Utiliser **gRPC (HTTP/2)** pour la communication entre les boîtiers GPS et le Service Localisation, ainsi qu'entre le Service Localisation et l'API Gateway pour l'affichage temps réel. Les opérations classiques conservent REST ou GraphQL.

Protocole	Usage
gRPC (server-streaming)	Positions GPS temps réel boîtier → Service Localisation → Frontend
GraphQL	Requêtes agrégées du frontend (dashboard, historique)
REST/JSON	CRUD véhicules, conducteurs, maintenances

Conséquences

Avantages : latence très faible grâce à la connexion persistante HTTP/2, sérialisation binaire (Protobuf) plus légère que JSON, adapté au streaming unidirectionnel continu.

Inconvénients : les navigateurs web ne supportent pas gRPC nativement — une couche de traduction `grpc-web` ou un proxy Envoy est nécessaire entre le frontend et le service.

ADR-003 — Apache Kafka pour la communication asynchrone

Statut : **Accepté**

Contexte

Plusieurs actions dans le système DHL doivent déclencher des réactions dans d'autres services sans les bloquer : une alerte géofencing ne doit pas ralentir le traitement des positions GPS, et la fin d'une maintenance doit automatiquement remettre le véhicule en disponibilité. Une communication synchrone directe entre services créerait un couplage fort et des risques de cascade en cas de panne.

Décision

Utiliser **Apache Kafka** comme bus d'événements central. Chaque service publie des événements sur des topics dédiés et consomme ceux qui le concernent, sans rien savoir des autres services. Six topics sont définis :

Topic	Rôle
`flotte.vehicules.events`	Changements d'état et de statut des véhicules
`flotte.conducteurs.events`	Cycle de vie des livreurs, expiration des permis
`flotte.assignments.events`	Affectation véhicule ↔ livreur par tournée
`flotte.localisation.gps`	Positions GPS en temps réel (haute fréquence)
`flotte.maintenance.events`	Révisions planifiées, en cours ou terminées
`flotte.alertes.events`	Alertes consolidées vers le manager et le frontend

Conséquences

Avantages : découplage total entre services, aucune perte de message (rétention configurable), rejeu possible en cas d'erreur, scalabilité horizontale via les partitions.

Inconvénients : composant d'infrastructure supplémentaire à déployer et surveiller. La cohérence des données devient éventuelle (*eventual consistency*), ce qui est acceptable pour les cas d'usage du projet.

ADR-004 — Bases de données spécialisées par service

Statut : **Accepté**

Contexte

Le système manipule trois catégories de données aux caractéristiques très différentes. Les positions GPS s'accumulent à haute fréquence et nécessitent des requêtes temporelles efficaces. Les données métier sont relationnelles et transactionnelles. Certaines données très consultées doivent être accessibles en quelques millisecondes.

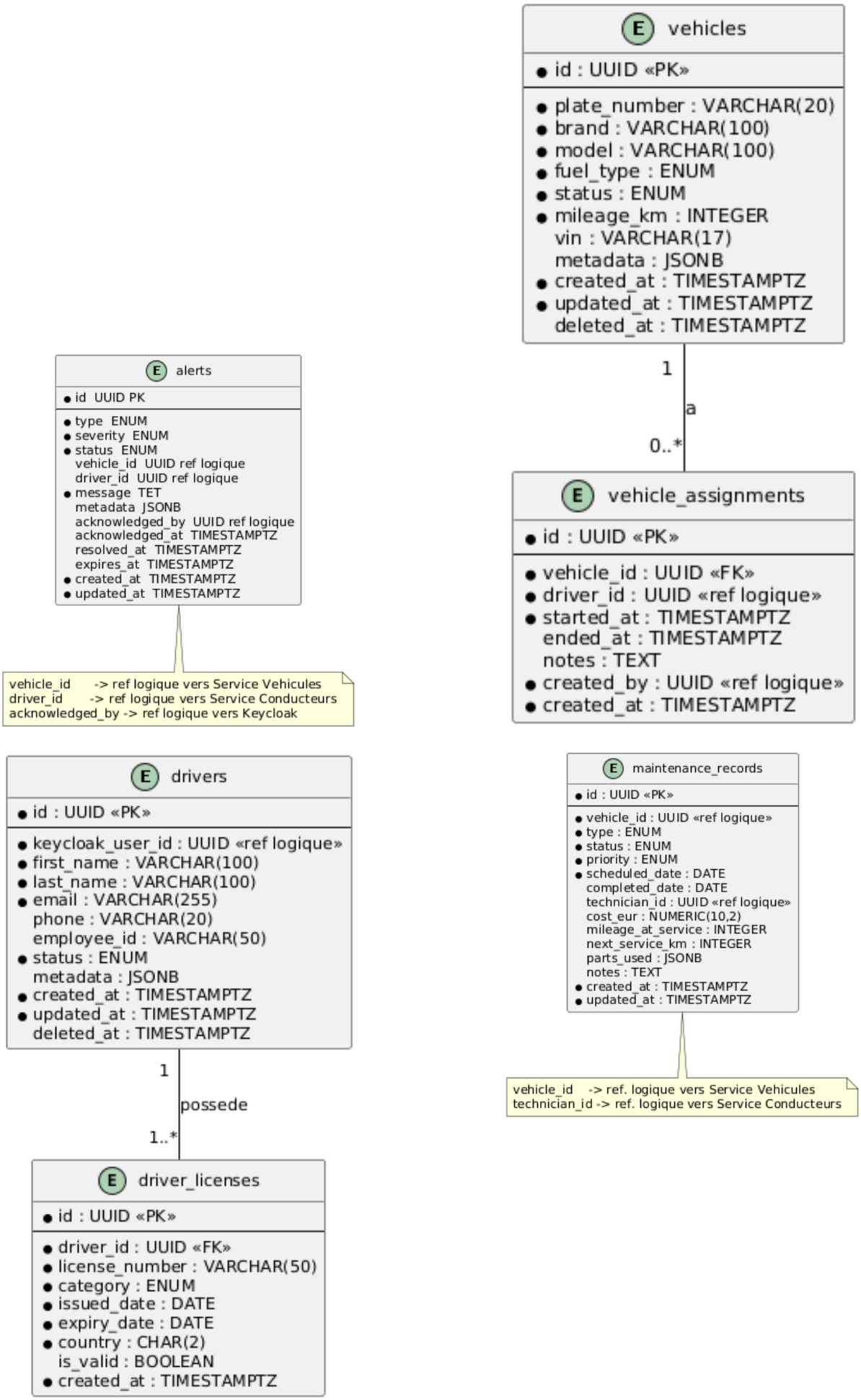
Décision

Appliquer le pattern **Database per Service** avec trois outils complémentaires :

Outil	Type	Rôle dans le projet DHL
PostgreSQL	Relationnel	Véhicules, livreurs, maintenances, alertes
TimescaleDB	Séries temporelles	Historique des positions GPS (`location_readings`)
Redis	Cache en mémoire	État courant des véhicules, sessions, données fréquentes

Le schéma PostgreSQL est composé de 6 tables : `vehicles`, `vehicle_assignments`, `drivers`, `driver_licenses`, `maintenance_records` et `alerts`. Ce périmètre est volontairement limité pour garantir un MVP fonctionnel.

Schémas de base de données (PostgreSQL)


vehicles
id : UUID «PK»
plate_number : VARCHAR(20)
brand : VARCHAR(100)
model : VARCHAR(100)
fuel_type : ENUM
status : ENUM
mileage_km : INTEGER
vin : VARCHAR(17)
metadata : JSONB
created_at : TIMESTAMPTZ
updated_at : TIMESTAMPTZ
deleted_at : TIMESTAMPTZ
1
a
0..*
vehicle_assignments
id : UUID «PK»
vehicle_id : UUID «FK»
driver_id : UUID «ref logique»
started_at : TIMESTAMPTZ
ended_at : TIMESTAMPTZ
notes : TEXT
created_by : UUID «ref logique»
created_at : TIMESTAMPTZ
alerts
id UUID PK
type ENUM
severity ENUM
status ENUM
vehicle_id UUID ref logique
driver_id UUID ref logique
message TET
metadata JSONB
acknowledged_by UUID ref logique
acknowledged_at TIMESTAMPTZ
resolved_at TIMESTAMPTZ
expires_at TIMESTAMPTZ
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
vehicle_id -> ref logique vers Service Vehicules
driver_id -> ref logique vers Service Conducteurs
acknowledged_by -> ref logique vers Keycloak
drivers
id : UUID «PK»
keycloak_user_id : UUID «ref logique»
first_name : VARCHAR(100)
last_name : VARCHAR(100)
email : VARCHAR(255)
phone : VARCHAR(20)
employee_id : VARCHAR(50)
status : ENUM
metadata : JSONB
created_at : TIMESTAMPTZ
updated_at : TIMESTAMPTZ
deleted_at : TIMESTAMPTZ
1
possede
1..*
driver_licenses
id : UUID «PK»
driver_id : UUID «FK»
license_number : VARCHAR(50)
category : ENUM
issued_date : DATE
expiry_date : DATE
country : CHAR(2)
is_valid : BOOLEAN
created_at : TIMESTAMPTZ
maintenance_records
id : UUID «PK»
vehicle_id : UUID «ref logique»
type : ENUM
status : ENUM
priority : ENUM
scheduled_date : DATE
completed_date : DATE
technician_id : UUID «ref logique»
cost_eur : NUMERIC(10,2)
mileage_at_service : INTEGER
next_service_km : INTEGER
parts_used : JSONB
notes : TEXT
created_at : TIMESTAMPTZ
updated_at : TIMESTAMPTZ
vehicle_id -> ref. logique vers Service Vehicules
technician_id -> ref. logique vers Service Conducteurs

Conséquences

Avantages : chaque outil est optimisé pour son type de données, pas de contention entre charges différentes, TimescaleDB réduit la taille des données GPS grâce à la compression automatique.

Inconvénients : trois systèmes à opérer et sauvegarder. Les jointures entre données de services différents sont réalisées au niveau applicatif via GraphQL.

ADR-005 — Keycloak pour l'authentification et les droits

Statut : **Accepté**

Contexte

Les utilisateurs du système DHL n'ont pas tous accès aux mêmes fonctionnalités. Un manager peut superviser le parc mais ne peut pas modifier les données de maintenance. Un technicien accède aux fiches d'intervention mais pas aux données des autres utilisateurs. Implémenter cette logique dans chaque microservice serait coûteux à maintenir et source d'incohérences.

Décision

Centraliser l'authentification et la gestion des droits dans **Keycloak** (SSO open-source). Chaque requête passe par l'API Gateway qui valide le **JWT** émis par Keycloak avant de router vers le bon service.

Rôle Keycloak	Droits dans le système DHL
`admin`	Accès complet à tous les services et données, gestion des comptes
`manager`	Supervision du parc, validation des maintenances, gestion des tournées
`technicien`	Lecture/écriture sur les fiches de maintenance, consultation des véhicules
`utilisateur`	Consultation de ses propres données (véhicule assigné, informations personnelles)

Les microservices ne gèrent **aucun mot de passe** directement. Ils vérifient uniquement la signature du JWT et extraient le rôle pour appliquer les autorisations (RBAC).

Conséquences

Avantages : gestion des droits centralisée et cohérente, SSO (connexion unique pour toutes les parties du système), standard OAuth2/OIDC.

Inconvénients : Keycloak est un point critique — s'il est indisponible, aucune authentification n'est possible. Il doit être déployé en mode haute disponibilité (minimum 2 réplicas Kubernetes).

ADR-006 — Observabilité avec OpenTelemetry

Statut : **Accepté**

Contexte

Dans une architecture microservices, une requête utilisateur traverse plusieurs services avant de retourner une réponse. En cas d'erreur ou de lenteur, il est très difficile d'identifier quel service est responsable sans outillage dédié. Les métriques opérationnelles et métier sont également indispensables au pilotage du système DHL.

Décision

Adopter **OpenTelemetry** comme standard unique d'instrumentation, couvrant les trois piliers de l'observabilité :

Pilier	Outil	Usage dans le projet
Traces	Jaeger	Suivi distribué d'une requête à travers tous les services
Métriques	Prometheus + Grafana	Tableaux de bord opérationnels et métier (véhicules actifs, alertes)
Logs	Loki	Agrégation centralisée des logs structurés JSON de tous les services

Chaque service est instrumenté via le SDK OpenTelemetry et exporte vers un **OTel Collector** central. Un `correlation_id` est propagé dans tous les messages Kafka et requêtes HTTP pour relier les traces bout-en-bout.

Conséquences

Avantages : standard ouvert (pas de vendor lock-in), instrumentation mutualisée, corrélation logs/traces/métriques facilitée, Grafana unifie la visualisation.

Inconvénients : légère surcharge mémoire et CPU liée à l'instrumentation. Configuration initiale du Collector à prévoir en semaine 2.

Synthèse des décisions — Les 6 ADR ci-dessus définissent les fondations techniques du projet. L'approche retenue privilégie un **MVP fonctionnel** : périmètre de base de données volontairement simple (6 tables), stack éprouvée, enrichissements progressifs semaine après semaine.

Configuration Keycloak — Semaine 1

Cette section documente les étapes de mise en place de Keycloak réalisées en semaine 1, constituant le livrable de sécurité du projet.

1. Déploiement du serveur Keycloak en local

Pour l'environnement de développement, Keycloak a été conteneurisé et lancé via Docker avec la création automatique d'un compte super-administrateur. La commande suivante démarre Keycloak en mode développement sur le port 8080 :

```
docker run -p 8080:8080
  -e KEYCLOAK_ADMIN=admin
  -e KEYCLOAK_ADMIN_PASSWORD=admin
  quay.io/keycloak/keycloak:latest start-dev
```

L'interface d'administration est ensuite accessible sur `http://localhost:8080`.

2. Création du Realm

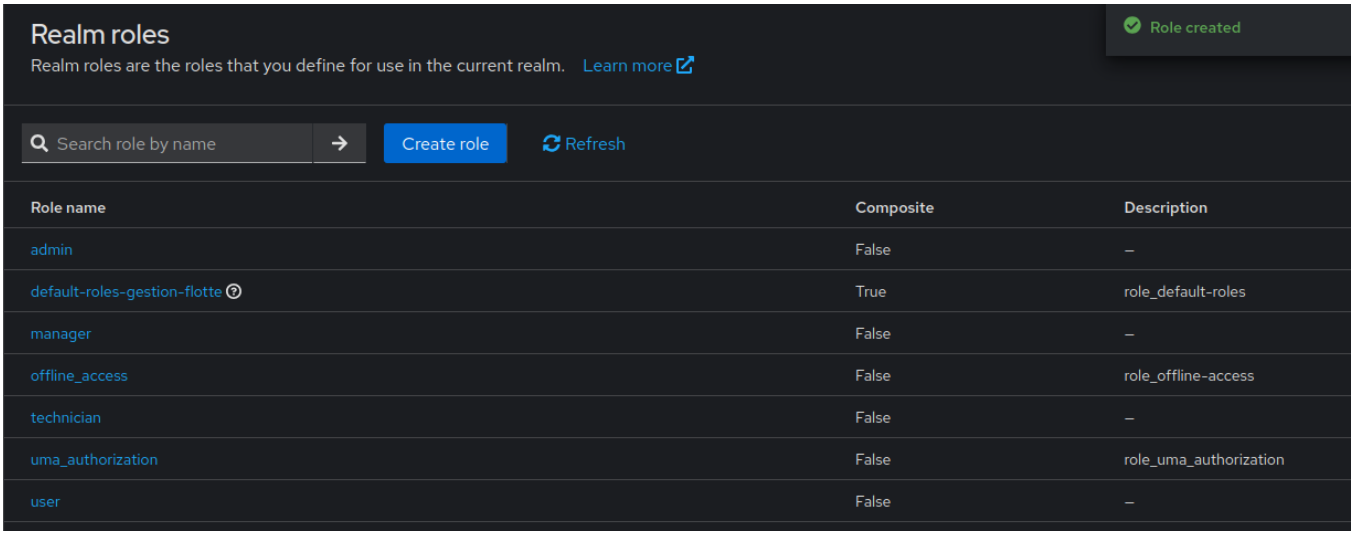
Afin d'isoler notre environnement du realm `master` (réservé à l'administration de l'outil), un nouveau Realm dédié au projet a été créé :

- **Nom du Realm :** `gestion-flotte`

3. Mise en place du contrôle d'accès (RBAC)

Les rôles métier suivants ont été définis dans la section *Realm roles* :

Rôle	Périmètre
`admin`	Accès complet à tous les services et données
`manager`	Supervision du parc, validation des maintenances
`technicien`	Lecture/écriture sur les fiches de maintenance
`user`	Consultation de ses propres données uniquement



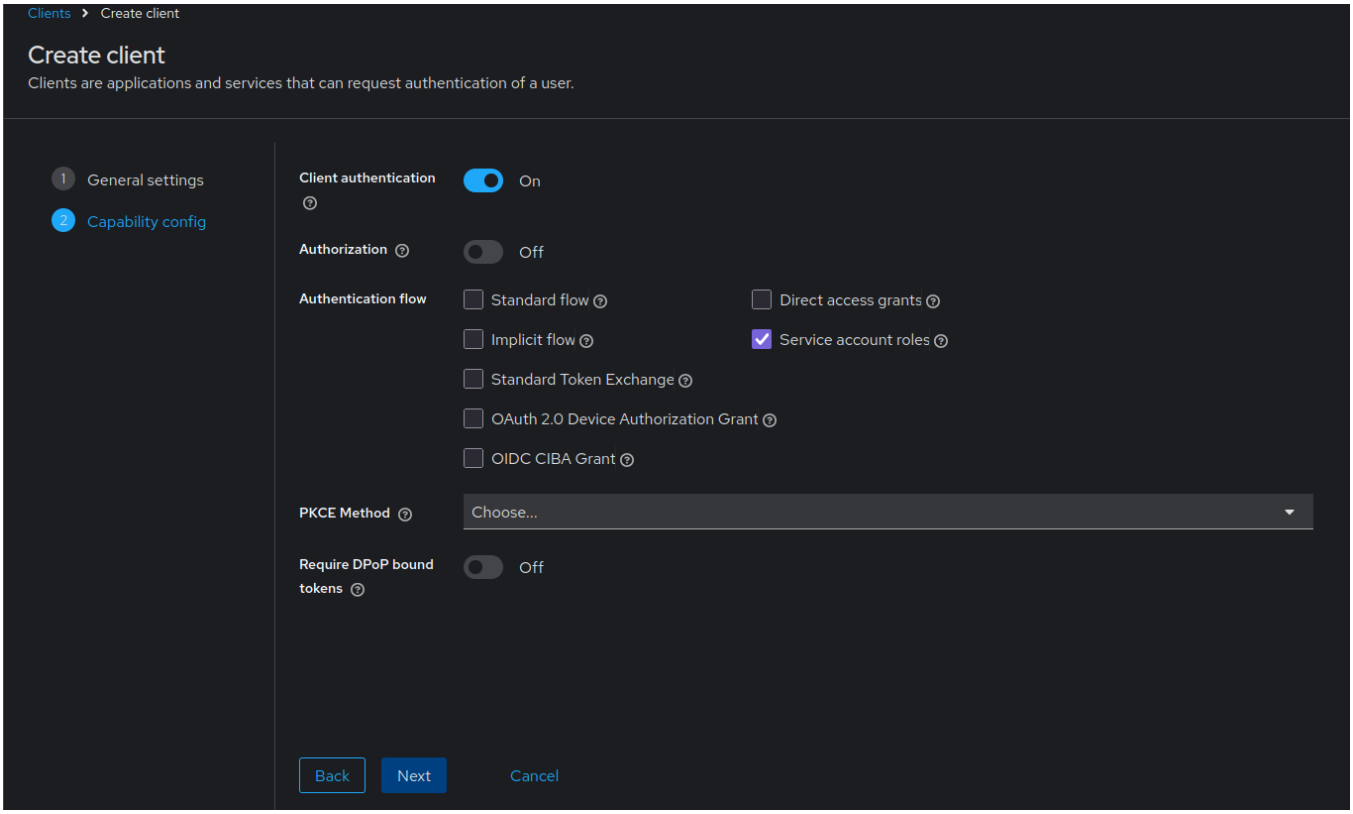
Liste des Realm roles créés dans le realm `gestion-flotte`

4. Configuration des Clients

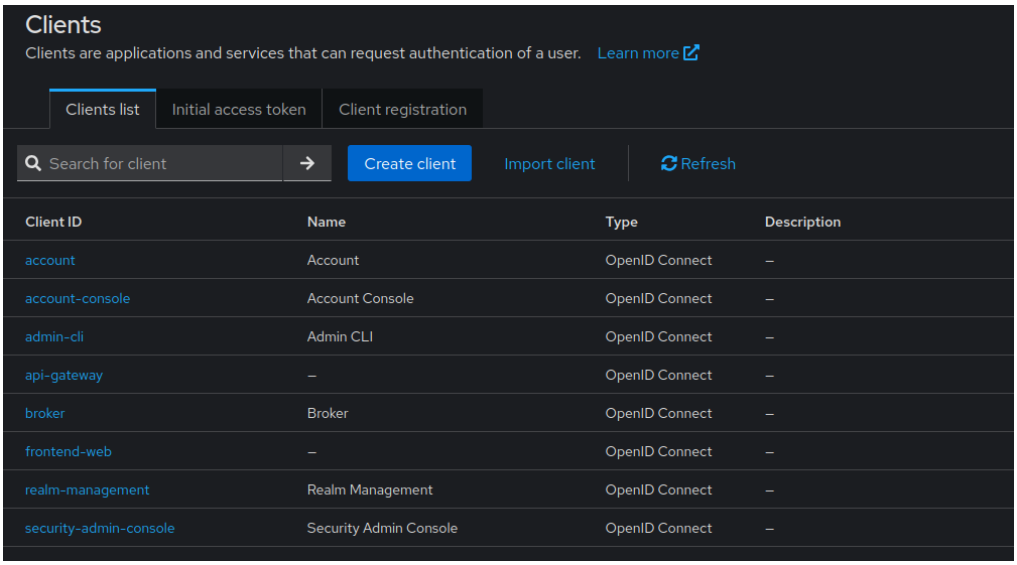
Deux clients ont été configurés pour interagir avec Keycloak :

Client 1 : `frontend-web` **(Client Public)** — Destiné à l'interface React/Vue. Configuration : Standard flow activé, redirection autorisée vers $*$ pour le développement.

Client 2 : `api-gateway` **(Client Confidentiel)** — Destiné à la communication serveur à serveur. Configuration : *Client authentication* activée, *Service accounts roles* activés, génération d'un *Client Secret* pour sécuriser les échanges GraphQL.



Liste des clients configurés dans le realm `gestion-flotte`

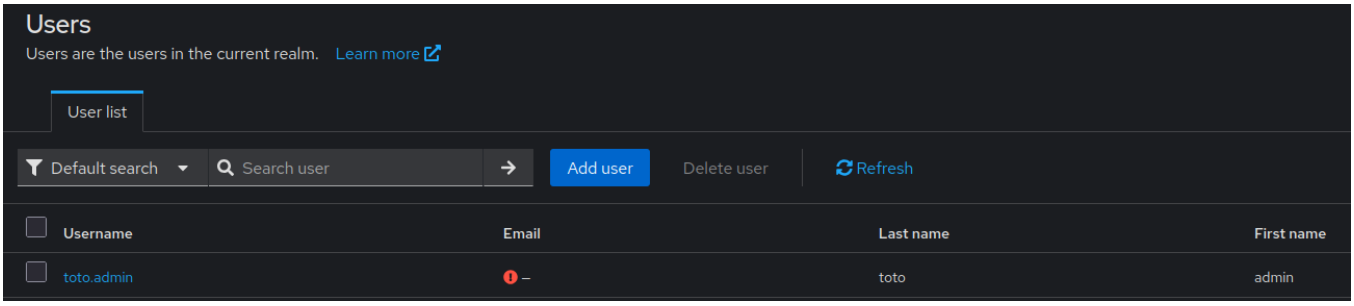


Configuration du client `api-gateway` — Client authentication ON, Service account roles activé

5. Création d'un utilisateur de test

Un utilisateur de test a été provisionné pour valider les futurs développements :

Paramètre	Valeur
Identifiant	`toto.admin`
Mot de passe	Défini manuellement, *Temporary = OFF*
Role mapping	Rôle `admin` assigné



Liste des utilisateurs du realm `gestion-flotte` — utilisateur `toto.admin` créé avec le rôle `admin`

6. Exportation de la configuration (Livrable)

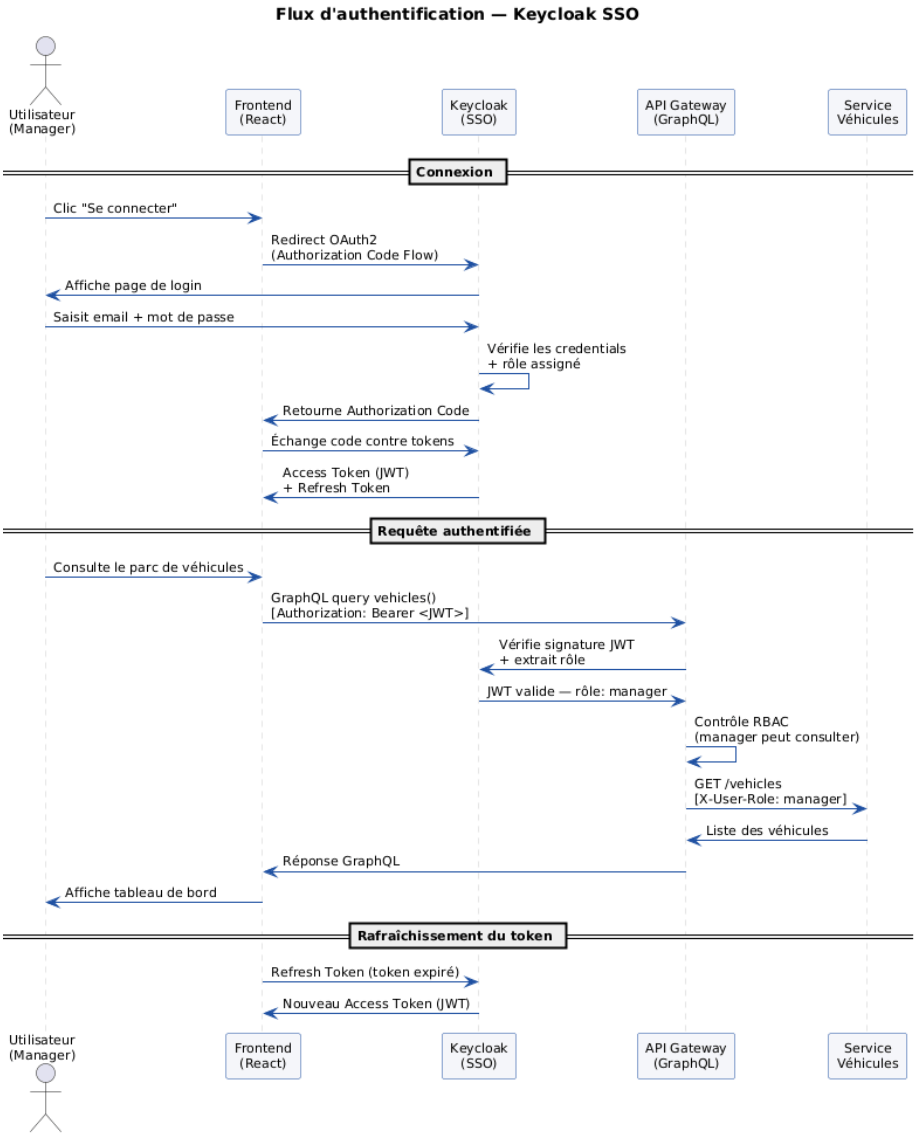
Pour garantir la reproductibilité de l'environnement (*Infrastructure as Code*), l'intégralité de la configuration a été exportée via l'interface Keycloak (Include clients + Include groups and roles). Le fichier généré `realm-export.json` est versionné dans le dépôt Git du projet sous le chemin `infrastructure/keycloak/realm-export.json`, ce qui permet de reconstruire l'environnement Keycloak complet automatiquement lors du déploiement en semaine 2.

Diagrammes de séquence — Flux principaux

Cette section illustre les trois flux techniques les plus importants du système DHL, modélisés sous forme de diagrammes de séquence PlantUML.

Flux 1 — Authentification Keycloak (OAuth2)

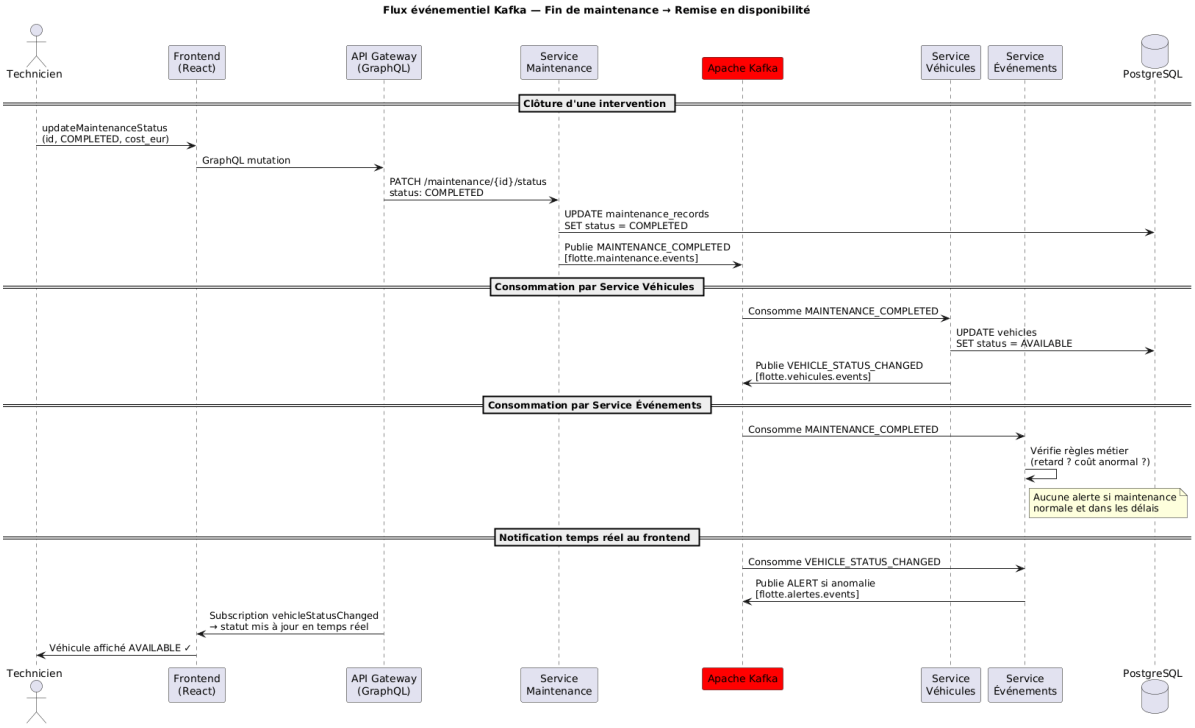
Ce diagramme décrit le flux complet de connexion d'un utilisateur via Keycloak, depuis le clic sur "Se connecter" jusqu'à l'affichage du tableau de bord. Il couvre la validation du JWT par l'API Gateway et le rafraîchissement automatique du token.



Flux d'authentification SSO — Authorization Code Flow OAuth2/OIDC

Flux 2 — Kafka : fin de maintenance et remise en disponibilité

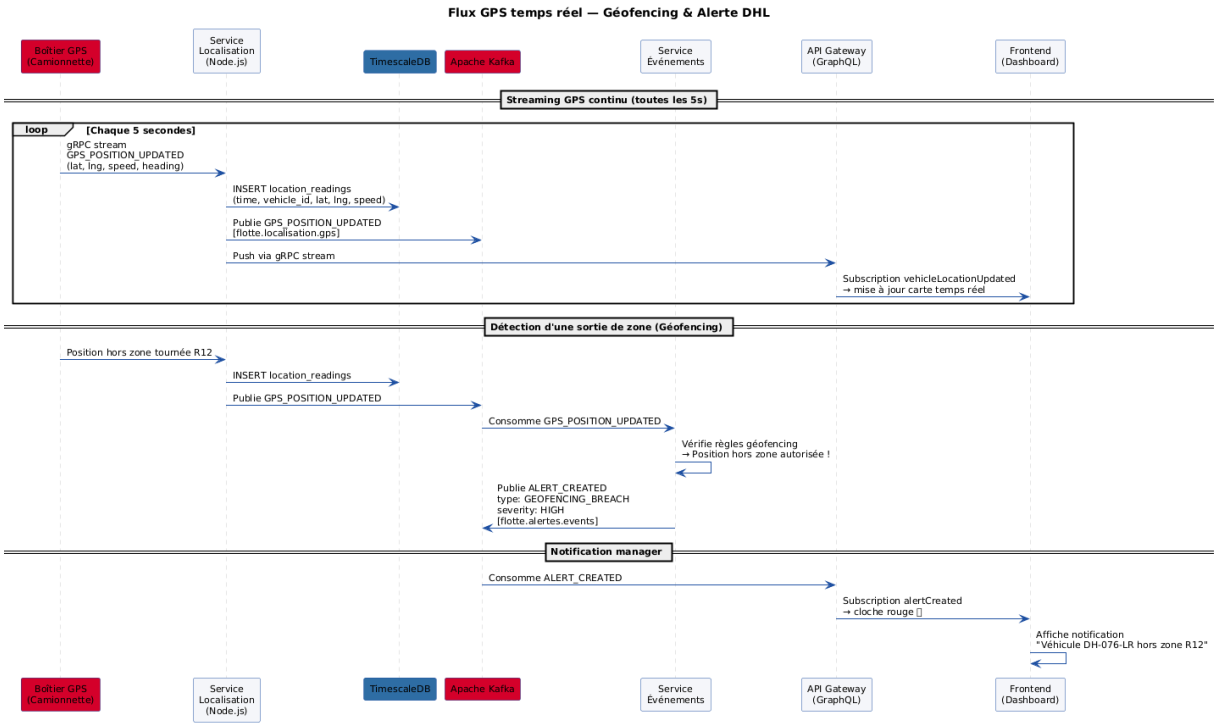
Ce diagramme illustre le flux événementiel déclenché lorsqu'un technicien clôture une intervention. L'événement Kafka est consommé par le Service Véhicules (remise en statut `AVAILABLE`) et par le Service Événements (vérification d'anomalies).



Flux événementiel Kafka — pattern publish/subscribe entre microservices

Flux 3 — GPS temps réel et détection de géofencing

Ce diagramme présente le flux de streaming GPS depuis le boîtier embarqué jusqu'au dashboard. Il détaille la chaîne de traitement d'une alerte géofencing : détection par le Service Événements, publication sur `flotte.alertes.events`, et notification temps réel via GraphQL Subscription.



Flux GPS temps réel — streaming gRPC et alerte géofencing DHL